

CS 498 RM - MP3 Calibration and Pose Estimation

Due on 11:59PM Friday, November 7, 2025

October 24, 2025

Q0 - Time Spent

Please record and report the number of hours you spend on completing this MP. This question will not be graded.

MP Description

In this MP, you will first calibrate the transformation between a wrist camera and the robot end-effector, then use the calibration result to estimate the pose of a vase, grasp it, and lift it off the table. Your calibration and pose estimation results will be considered accurate enough if the subsequent vase grasping and lifting achieves at least 90% success rate in 10 **consecutive** runs. You are **NOT allowed** to use the ground truth camera and vase poses directly obtained from the simulator utilities.

The provided starter code consists of four scripts (look for **TODO** segments in the scripts to add your code):

1. `task.envs.py`: This file contains the environment setup for the MP. You will use `MP3CalibEnv` / `MP3CalibSceneCfg` for hand-eye calibration and `MP3PoseEstimEnv` / `MP3PoseEstimSceneCfg` for vase pose estimation and grasping. The vase pose will be randomized every time the environment is initialized. You are **NOT allowed** to modify this file.
2. `keyboard_teleop.py`: This file provides a teleoperation function for you to move the robot around by pressing your keyboard. You will need to modify this file to collect the calibration images. Running `python3 keyboard_teleop.py` for robosuite and `python3 keyboard_teleop.py --enable_cameras` for Isaac Lab will start the teleoperation process. Once started, the keyboard control mapping will be printed in the terminal. You can remap the controls if you prefer.
3. `calibrate.py`: This file is where you will write the hand-eye calibration code. Running it should perform the computations needed for calibration and output the calibrated transformation matrix.
4. `pose_estimation.py`: This file is where you will write the vase pose estimation and grasping code. Running `python3 pose_estimation.py` for robosuite and `python3 pose_estimation.py --enable_cameras` for Isaac Lab will execute your code under the `run` function. You can write additional functions in the `Experiment` class to help you better complete the MP.

In your Canvas submission, please include the following:

1. A typed report that answers all questions in this document. The questions to be answered are in **bold font**.
2. A zip file of the entire MP code folder, including files we provided and files you modified/created.
3. A video demonstrating your method, as detailed in Q6.

If you referenced any public code segments from the internet, please clearly state the source in your report.

Important Hints

robosuite

1. For all images (RGB and depth) obtained from robosuite, you would need to flip them in the x-axis. You can do so using functions such as `cv2.flip()`.
2. Please use `camera_utils.get_real_depth_map(self.env.sim, self.obs[camera_name_depth])` to get the depth map. The default option, `obs[camera_name_depth]`, only returns normalized depth maps with values between 0 and 1.

Isaac Lab

1. `cv2.imshow()` might not work in Isaac Lab. We do not recommend trying to fix it by modifying the OpenCV installation, as this might break the entire Isaac Lab setup. You can use `matplotlib` as an alternative for image visualization.
2. Please use `gripper_center` as the end-effector frame.

Part 1 - Hand-Eye Calibration

In the first part of this MP, you will complete hand-eye calibration to estimate the transformation matrix between the robot wrist camera (`robot0_eye_in_hand` in robosuite and `wrist_camera` in Isaac Lab) and the robot end-effector. The general hand-eye calibration procedure is described in lecture 11.

Q1 - AprilTag Pose Estimation

In `keyboard_teleop.py`, please complete the code for AprilTag detection and pose estimation. You are allowed to use external libraries such as `pupil_apriltags`. The AprilTag used in this MP is from the tag family `tag36h11` and has a size of 0.16m. The camera intrinsics are provided in the `intrinsics` variable. **Please attach an image visualizing the detected AprilTag and its estimated pose.**

Q2 - Hand-Eye Calibration

Please modify `keyboard_teleop.py` to collect the hand-eye calibration dataset. As you move the robot to different poses where the AprilTag is detectable, record the AprilTag image/pose and the current robot end-effector pose. After collecting your dataset, you are allowed to use external Python libraries to solve for the hand-eye transformation matrix. In our basic implementation, we were able to achieve a decent calibration result using 15 calibration images. **Please report the hand-eye transformation matrix (camera pose in the robot end-effector frame) you obtained. How did you choose the robot poses for collecting calibration images?**

Part 2 - Pose Estimation

Suppose we have the ground truth 3D model of a vase, and we have pre-computed a valid grasping pose located on the vase. The vase origin and the grasping pose are shown in Figure 1. The ground truth point cloud of the vase shown in Figure 1 is provided in `/objects/vase.npz`, and the pre-computed grasping pose is provided in `T_vase_grasp.npz`. If we can estimate the pose of the vase in `MP3PoseEstimEnv` / `MP3PoseEstimSceneCfg`, we can transform the vase grasping pose into the world frame and command the robot to perform grasping. We will perform pose estimation by fitting the point cloud in `vase.npz` to the perceived vase point cloud in `MP3PoseEstimEnv` / `MP3PoseEstimSceneCfg` using Iterative Closest Point (ICP). Please refer to lecture 12 for details on pose estimation and ICP.

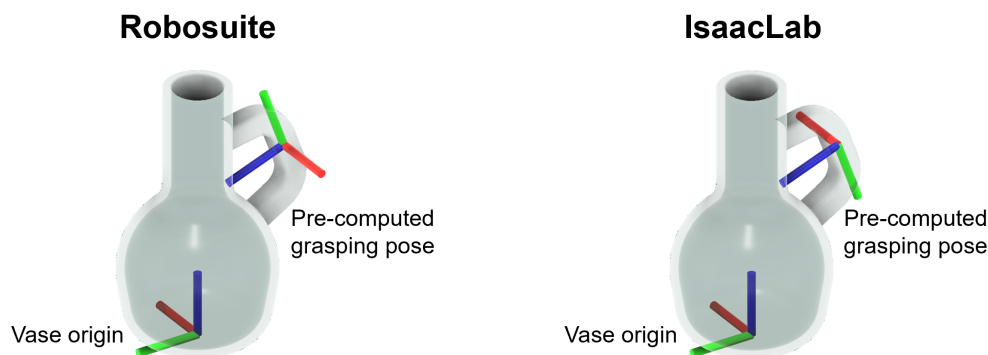


Figure 1: Vase origin and grasping code in the two simulators.

Q3 - Object Point Cloud Reconstruction

We first need to obtain the point cloud of the vase in `MP3PoseEstimEnv` / `MP3PoseEstimSceneCfg`. To do so, follow the steps below:

1. Use the robot wrist camera (`robot0_eye_in_hand` in robosuite and `wrist_camera` in Isaac Lab) to capture the RGBD image of the vase.
2. Deproject the depth image into a point cloud.
3. Filter out the vase point cloud with color thresholding.
4. Use the hand-eye transformation matrix you obtained in Part 1 and the robot end-effector pose to transform the vase point cloud into the world coordinate frame.

Since ICP is sensitive to partial views, you might want to move the robot around to capture multiple depth images from different perspectives and obtain a more complete point cloud for the vase. **Please take some screenshots to indicate the poses you are capturing the depth images from and attach them to the report. How did you choose these poses? Please also attach an image of the combined vase point cloud.**

Q4 - Iterative Closest Point

Use the Iterative Closest Point algorithm to register the point cloud in `vase.npz` to the perceived point cloud from Q3. **In your report, briefly describe how ICP works.** You are allowed to use existing Python libraries to perform ICP, or you can implement ICP from scratch for extra credit up to 5% the total available points from this MP. **Please attach an image of the ICP registration result by visualizing the source and target point clouds before and after ICP. Please clearly label the source and target point clouds with different colors.**

Q5 - Transform Grasp Pose

Now that we have the estimated transformation between the robot wrist camera and the robot end-effector, we can transform the pre-computed grasping pose into the environment world frame. **How can we find the pose of the vase and the grasping pose (both in the environment world frame) from the ICP registration result? Please write down the equations and briefly explain.** Note that you don't need to report the numerical values, but please clearly label what each transformation matrix in the equation represents.

Q6 - Report Results

Please run and record your pose estimation and grasping pipeline 10 consecutive times and report the success rate. Please compile your recordings into one video and include it in your Canvas submission. In your video, please display proper labels, such as the trial number and the result of the current trial. A grasping success rate of 90% or above will grant you full credit. A run is considered successful if it satisfies the following criteria:

1. The robot grasps onto the vase at the pre-computed grasping pose.
2. The vase does not slip out of the robot gripper for at least 2 seconds after being lifted. Please note that in robosuite, the vase might have a tendency to slip out of the robot gripper after being lifted. It is acceptable if the vase is slipping in the robot gripper, as long as it remains in the gripper for at least 2 seconds.

In your report, please analyze: **What are the failure modes of your pipeline (if any)? How can it be improved?**

Extra Credit

Completing the task(s) described below will grant you extra credit up to 25% of the total available points from this MP. If you attempted the extra credit task but failed to finish it, feel free to describe and showcase what you have done, and we might give you partial extra credit based on your results. **We might or might not have verified whether it is actually possible to complete the extra credit task(s), and we cannot guarantee the amount of extra credit we offer is proportional to the time required to complete the extra credit task(s). Please invest your time accordingly.**

There are two extra credit opportunities for this MP:

1. (5%) Implement ICP from scratch, as mentioned in Q4.
2. (20%) Find an existing paper or library estimating valid grasp poses from object point cloud and/or RGB images and incorporate it into the grasping portion of this MP. You can run the code provided by the paper or implement it yourself. **Briefly describe the method proposed in the paper/library you chose, visualize the valid grasp poses computed, and repeat the steps in Q6 for reporting. If the method does not work well, briefly discuss why that is the case.**